

CHEEM: Continual Learning by Reuse, New, Adapt and Skip

A Hierarchical Exploration-Exploitation Approach

Chinmay Savadikar, Mengyue Dai, and Tianfu Wu · CVPR 2026

2026.07

Hyuntaek Seo

Agenda

1. **Problem Definition** — ExfCCL Constraints, Core Dilemma, and Extreme Data Imbalance
2. **Proposed Solution (CHEEM)** — Decoupled Memory, Primitive Operations, and HEE-NAS
3. **Comparative Analysis** — Evaluation Metric (FoM), SOTA Comparisons, and Ablation Studies

1. Problem Definition: The Stability-Plasticity Dilemma

The Core Conflict

- **Stability:** Retaining previously acquired knowledge
- **Plasticity:** Learning new incoming task features quickly.
- **Catastrophic Forgetting:** Updating shared weights for new tasks overwrites parameters optimized for past tasks.

ExfCCL Setting Constraints

- **Exemplar-free:** Past task data/samples are completely inaccessible.
- **Class-Incremental:** Category count grows continuously.
- **Task-ID Unknown:** Model must predict task ID at inference to select the classifier head.

1. Problem Definition: Extreme Real-World Imbalance

Existing CL methods rely on synthetic, balanced datasets. CHEEM addresses highly skewed domain streams.

Benchmark	Dataset Stream	Extreme Imbalance Scenario
MTIL <i>(Domain Stream)</i>	ImageNet → 11 streaming domains	• MNIST (54,000 images, 10 classes) vs. • VGG-Flowers (1,000 images, 102 classes)
VDD <i>(Class Imbalance)</i>	2,128 Total Category Stream	• Omniglot (1,623 classes) vs. • DTD (47 classes)

Bottleneck: Under this skewness, fixed-capacity networks either waste computation on easy tasks or suffer from representation collapse on hard/large ones.

1. Problem Definition: Why Existing Baselines Fall Short

Regularization (EWC)

- **Approach:** Penalizes weight updates on a single shared backbone.
- **Fail Case:** Fails to scale under extreme task shifts; severely suffers from catastrophic forgetting.

Prompting (L2P / CODA-P)

- **Approach:** Keeps backbone frozen, learns task-specific prompts.
- **Fail Case:** Fails on smaller backbones (DEiT-Tiny) which lack initial representational capacity.

Static PEFT (LoRA-CL)

- **Approach:** Injects LoRA adapters into all layers uniformly.
- **Fail Case:** Cannot dynamically skip layers for easy tasks, leading to high FLOPs overhead.

2. Proposed Solution: CHEEM Decoupled Memory

To address these limitations, CHEEM introduces two decoupled memory structures.

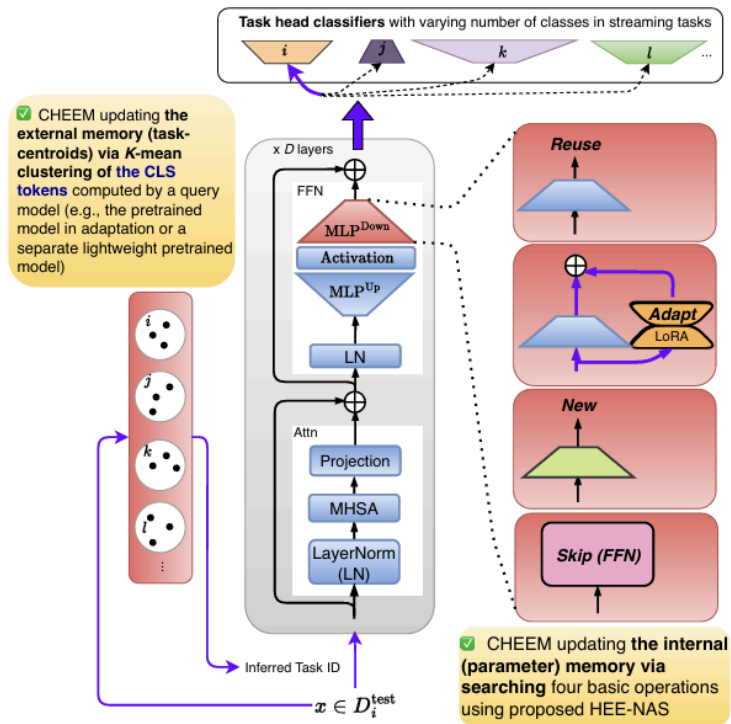


Figure 2: CHEEM Architecture overview

① External Task-Centroid Memory

- **Task ID Inference:** Performs K-Means clustering on CLS tokens.
- At inference, the input's CLS token is compared to stored task-centroids to predict the Task ID and route to the correct head.

② Internal Parameter Memory

- **Adaptive Parameter Allocation:** Uses HEE-NAS to dynamically adapt ViT components.
- Targets the **MLPDown** layer in FFN blocks to keep parameter overhead minimal.

2. Proposed Solution: Four Primitive Operations

CHEEM's internal memory dynamically selects block-by-block routes using four operators.

1. Reuse

Uses previous task's weights.
Zero parameter growth, maximizes knowledge transfer.

2. Adapt

Applies a task-specific LoRA.
Fine-tunes representation while preserving old weights.

3. New

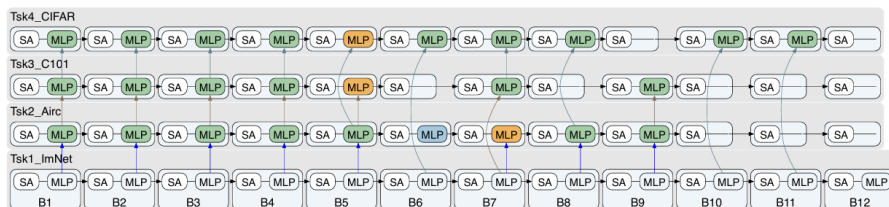
Installs a new parallel layer.
Enables learning of highly out-of-domain task features.

4. Skip

Bypasses the layer entirely.
Saves massive FLOPs on easy or redundant blocks.

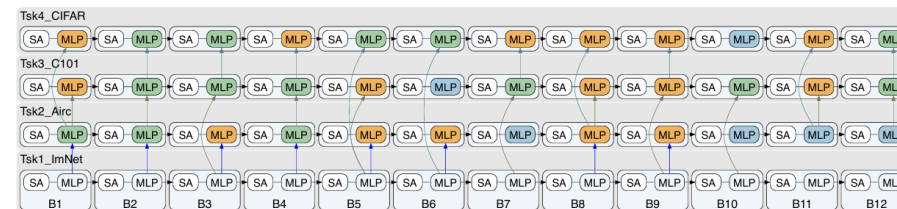
2. Proposed Solution: Task-Difficulty Adaptation

The HEE-NAS search space produces distinct paths depending on backbone size and task difficulty.



(b) From **ViT-Base trained on Tsk1.ImNet** (with blocks B1 to B12), our CHEEM learns sensible task-tailored models that reflect the task complexity. For example, when learning Caltech 101 (Tsk3.C101), CHEEM learns to **Skip** 5 MLP blocks and **Reuse** most of the architecture. On the contrary, when learning FGVC Aircraft (Tsk1.Airc), which is a more complex task with larger shift from ImageNet due to its fine-grained nature, CHEEM learns to **Adapt** the ImageNet parameters in Block 7, adds a **New** operation in Block 6, and **Skips** the last 3 MLP blocks. See text for details. Full structure reproduced in Fig. 7 in the supplementary.

ViT-Base (Large Backbone): Caltech 101 (Easy) skips 5 FFN blocks; Aircraft (Hard) deploys New/Adapt.



(c) From **DEiT-Tiny trained on Tsk1.ImNet** (with blocks B1 to B12), our CHEEM learns to use multiple **Adapt** and **New** operations, without **Skip** operations selected, sensibly different from those with more **Skip** and less **New** operations learned based on the stronger ViT-Base model. Full structure reproduced in Fig. 9 in the supplementary.

DEiT-Tiny (Small Backbone): Lacks capacity; rarely skips, actively adds New/Adapt to grow depth/width.

2. Proposed Solution: HEE-NAS Search Mechanism

HEE-NAS automates neural architecture search by prioritizing task synergy (similarity).

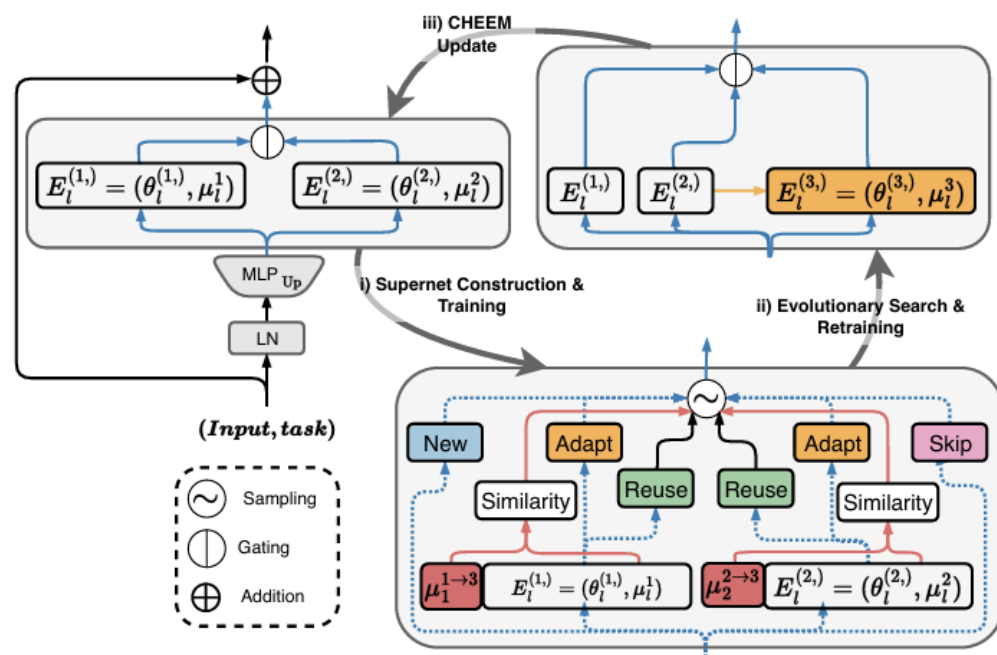


Figure 4. Illustration of CHEEM learning via NAS.

Figure 4: HEE-NAS evolution pipeline

3-Step Pipeline

- Supernet Construction & Training:** Builds a search space comprising the 4 operations (Reuse, Adapt, New, Skip).
- HEE Evolutionary Search:** Searches for Pareto-optimal structures under accuracy vs. FLOPs constraints.
- CHEEM Update:** Selects the best structure to update internal parameter memory, and retrain the model.

2. Proposed Solution: Task Similarity & Sampling Equations

HEE sampling uses cosine similarity of CLS tokens to guide exploration.

1. Similarity Calculation

- **Task Similarity** ($S_{i,t}^l$): Cosine similarity between old task i 's original token and new task t 's projected token at block l .

$$S_{i,t}^l = \text{NormCosine}(\mu_l^i, \mu_l^{i \rightarrow t})$$

- **Auxiliary Expert** ($S_{aux,t}^l$): Safety valve when task similarity is extremely low (drives New / Skip).

$$S_{aux,t}^l = -\max_{i=1}^{t-1} S_{i,t}^l$$

2. Hierarchical Sampling

- **Level 1 (Knowledge Selection)**: Categorical probability distribution (ψ) over old tasks and Aux.

$$\psi_i^l = \text{Softmax}(S_{:,t}^l)_i = \frac{\exp(S_{i,t}^l)}{\sum_k \exp(S_{k,t}^l)}$$

- **Level 2 (Operator Routing)**: Bernoulli trial probability (ρ_i^l) for task i .

$$\rho_i^l = \text{Sigmoid}(S_{i,t}^l)$$

- **Reuse** with prob ρ_i^l , **Adapt** with prob $1 - \rho_i^l$.
- If Aux is chosen, select **New** or **Skip** (50/50).

2. Proposed Solution: Search Space Scheduling

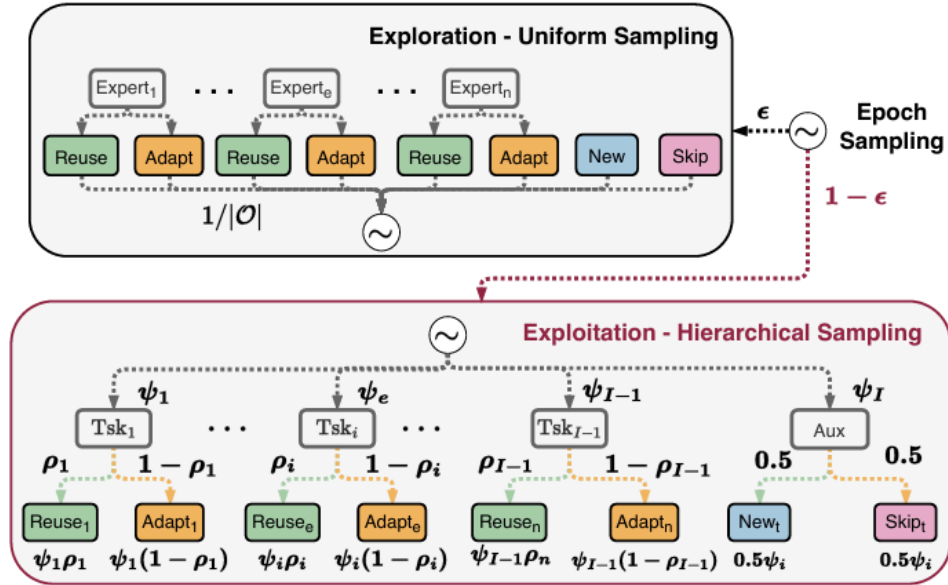


Figure 5. Illustration of the proposed HEE sampling based NAS. It integrates the vanilla exploration strategy (top) and the proposed exploitation strategy (bottom) with an epoch-wise scheduling.

Figure 5: Sampling-based search space

Exploration vs. Exploitation

To prevent local minima and discover novel configurations:

- **ϵ -Greedy Scheduling:**
 - With probability ϵ_1 (e.g. 0.3): Perform **Pure Exploration** (uniform random sampling over 4 operations).
 - With probability $1 - \epsilon_1$ (e.g. 0.7): Perform **Exploitation** (HEE similarity-based sampling).
- This accelerates training convergence while discovering irregular architectures.

3. Comparative Analysis: Standard Continual Learning Metrics

To evaluate knowledge retention and forgetting prevention, CHEEM uses two traditional metrics.

Average Accuracy (AA)

- **Definition:** The mean Top-1 accuracy across all learned tasks at the final training stage N .

$$AA = \frac{1}{N-1} \sum_{t=2}^N \text{Acc}(D_t^{\text{test}}; F_N, H_N)$$

- **Role:** Represents the overall final capability of the model.

Average Forgetting (AF)

- **Definition:** The average performance drop of each task from its historical peak to the final stage N .

$$AF = \frac{1}{N-2} \sum_{t=2}^{N-1} \left(\max_{j \in [t, N]} a_{j,t} - a_{N,t} \right)$$

- **Role:** Measures resistance to catastrophic forgetting. Closer to 0% means stronger retention.

3. Comparative Analysis: Figure-of-Merit (FoM) Metric

To compare algorithms across both accuracy and computational complexity, CHEEM proposes the FoM.

$$FoM(m, n) = \left(\frac{AA_{\text{UpperBound}} - AA_n}{AA_{\text{UpperBound}} - AA_m} \right) \cdot \left(\frac{\text{FLOPs}_n}{\text{FLOPs}_m} \right)$$

Accuracy Gap Ratio (Left)

- Compares accuracy gaps against the upper-bound (Full fine-tuning).
- If model m gets closer to upper-bound, the denominator shrinks $\rightarrow$ FoM increases.

Complexity Ratio (Right)

- Evaluates the computational cost (FLOPs).
- If model m uses fewer FLOPs than baseline n , the ratio increases $\rightarrow$ FoM increases.

3. Comparative Analysis: FoM Efficiency Gain (Table 1)

Table 1. **FoM** of CHEEM (MLP^{Down}) against baselines on **MTIL** and **VDD** benchmarks.

Method	MTIL		VDD	
	ViT-Base	DEiT-Tiny	ViT-Base	DEiT-Tiny
EWC	10.5	26.0	12.1	678.9
CODA-P	24.2	84.6	35.9	2811.3
DualP	27.4	73.7	34.1	2130.6
L2P	31.1	79.8	36.4	2431.6
S-Prompts	3.2	10.5	5.5	338.8
DIKI	3.6	6.4	7.8	370.8
LoRA-CL	1.7	5.7	1.5	73.6
Tuna	50.6	242.4	58.6	4538.8
Moal	2.38	7.64	8.63	462.8

Table 1: Figure-of-Merit (FoM) comparisons on MTIL and VDD

Highlight: CHEEM vs. SOTA

- **vs. L2P (Prompting):**
 - ViT-Base (MTIL): **31.1x** better
 - DEiT-Tiny (VDD): **2,431.6x** better
- **vs. LoRA-CL (No Search):**
 - ViT-Base (MTIL): **1.7x** better
 - DEiT-Tiny (VDD): **73.6x** better
- Proves HEE-NAS's dynamic parameter allocation value.

Why the thousand-fold multiplier? On DEiT-Tiny VDD, CHEEM achieves 76.18% accuracy (Upper-bound: 76.21%). The gap is 0.03%. Since the denominator is near-zero, the FoM value explodes.

3. Comparative Analysis: Accuracy vs. Upper-Bounds

How close does CHEEM get to the absolute performance limit?

Table 2. **CHEEM vs Upper-Bounds on MTIL** with three seeds.

Method	ViT-Base			DEiT-Tiny		
	Avg. Acc	Avg. Frgt.	FLOPs	Avg. Acc	Avg. Frgt.	FLOPs
UpperBound _{Full-FT}	88.1 $\pm$ 0.0	-	33.7	75.3 $\pm$ 0.1	-	2.2
UpperBound _{LoRA-FT}	87.4 $\pm$ 0.0	-	33.7	74.6 $\pm$ 0.1	-	2.2
CHEEM	85.9 $\pm$ 0.3	1.7 $\pm$ 0.1	62.3	74.5 $\pm$ 0.3	1.9 $\pm$ 0.0	4.5

Table 3. **CHEEM vs Upper-Bounds on VDD** with three seeds.

Method	ViT-Base			DEiT-Tiny		
	Avg. Acc	Avg. Frgt.	FLOPs	Avg. Acc	Avg. Frgt.	FLOPs
UpperBound _{Full-FT}	88.7 $\pm$ 0.1	-	33.7	76.21 $\pm$ 0.1	-	2.2
UpperBound _{LoRA-FT}	86.8 $\pm$ 0.1	-	33.7	76.30 $\pm$ 0.3	-	2.2
CHEEM	86.7 $\pm$ 0.2	0.4 $\pm$ 0.0	61.6	76.18 $\pm$ 0.1	1.0 $\pm$ 0.0	4.5

Table 2 & 3: CHEEM vs. Upper-Bound Fine-tuning

Core Takeaways

- **Virtually zero forgetting:**
 - On **MTIL**, CHEEM achieves **85.9%** (vs. 88.1% Upper-Bound) on ViT-Base.
 - On **VDD**, CHEEM matches the Upper-Bound exactly at **76.2%** on DEiT-Tiny.
- **Task ID Inference Cost:**
 - FLOPs are nearly doubled compared to single path.
 - Reason: Testing requires one initial forward pass of the backbone to infer the Task ID before executing the routed path.

3. Comparative Analysis: Detailed Baselines (Table 4 & 5)

Comparing absolute accuracy and FLOPs against traditional and prompting methods.

Table 4. Comparison of Average Accuracy and Forgetting Rate on the MTIL benchmark with three seeds.

Method	ViT-Base			DEiT-Tiny		
	Avg. Acc	Avg. Frgt.	FLOPs	Avg. Acc	Avg. Frgt.	FLOPs
EWC	44.6 ± 6.4	23.8 ± 6.5	33.7	35.3 ± 0.3	7.3 ± 0.6	2.2
CODA-P	40.2 ± 1.2	25.3 ± 1.8	70.3	5.6 ± 0.3	42.6 ± 0.8	5.0
DualP	33.8 ± 0.4	22.1 ± 0.4	70.3	30.9 ± 0.3	17.5 ± 0.3	5.0
L2P	26.6 ± 0.2	31.0 ± 0.3	70.3	23.2 ± 0.1	25.8 ± 0.4	5.1
S-Prompts	81.6 ± 0.4	1.6 ± 0.1	67.6	67.3 ± 0.4	1.8 ± 0.0	4.4
DIKI	76.4 ± 0.0	2.0 ± 0.0	42.5	67.6 ± 0.1	1.8 ± 0.0	2.8
LoRA-CL	84.7 ± 0.0	1.6 ± 0.1	68.2	71.1 ± 0.0	1.9 ± 0.0	4.5
Tuna	70.7 ± 0.1	17.4 ± 0.2	405.0	44.5 ± 0.5	30.2 ± 0.3	26.0
MoAL	78.3 ± 0.1	3.7 ± 0.1	33.7	63.8 ± 0.0	2.8 ± 0.0	2.2
CHEEM	85.9 ± 0.3	1.7 ± 0.1	62.3	74.5 ± 0.3	1.9 ± 0.0	4.5

Table 5. Comparison of Average Accuracy and Forgetting Rate on the VDD benchmark with three seeds.

Method	ViT-Base			DEiT-Tiny		
	Avg. Acc	Avg. Frgt.	FLOPs	Avg. Acc	Avg. Frgt.	FLOPs
EWC	44.0 ± 1.3	5.1 ± 1.1	33.7	33.7 ± 0.2	1.5 ± 0.1	2.2
CODA-P	24.9 ± 2.2	26.1 ± 0.8	70.3	1.1 ± 0.1	37.6 ± 0.4	5.0
DualP	28.1 ± 0.9	3.2 ± 0.5	70.3	19.4 ± 0.6	10.5 ± 0.5	5.0
L2P	23.9 ± 0.7	9.0 ± 0.6	70.3	11.5 ± 0.8	20.9 ± 1.7	5.1
S-Prompts	78.6 ± 0.1	0.4 ± 0.0	67.6	65.8 ± 0.3	0.9 ± 0.0	4.4
DIKI	65.9 ± 0.1	0.1 ± 0.0	42.5	58.3 ± 0.1	0.6 ± 0.0	2.8
LoRA-CL	86.0 ± 0.1	0.3 ± 0.0	68.2	74.0 ± 0.3	1.1 ± 0.0	4.5
Tuna	70.7 ± 0.1	14.9 ± 0.1	337.0	47.8 ± 0.2	22.9 ± 0.3	21.5
MoAL	56.7 ± 0.1	1.0 ± 0.0	33.7	47.9 ± 0.1	2.1 ± 0.0	2.2
CHEEM	86.7 ± 0.2	0.4 ± 0.0	61.6	76.18 ± 0.1	1.0 ± 0.0	4.5

Table 4 & 5: Detailed accuracy & FLOPs comparisons

1. Traditional EWC (Regularization)

- Lowest FLOPs, but accuracy collapses due to catastrophic forgetting (**44.58%** on ViT-Base MTIL).

2. SOTA Prompting (CODA-Prompt)

- Accuracy collapses on DEiT-Tiny (**18.73%** on MTIL) due to representation bottlenecks.

3. LoRA-CL (No Search)

- Comparable accuracy to CHEEM, but uses higher FLOPs because it cannot skip layers.

3. Task-Difficulty Awareness & Training Overhead (Table 6 & 7)

Evaluating dynamic FLOPs allocation and training efficiency.

Table 6. Task-wise FLOPs of CHEEM on MTIL using ViT-Base.

Airc	C101	CIFAR	DTD	ESAT	Flwr
62.8 ± 0.9	59.0 ± 0.9	64.0 ± 0.0	64.6 ± 0.9	57.8 ± 0.9	62.1 ± 0.1
F101	MNIST	Pets	Cars	SUN397	Avg
65.2 ± 1.7	56.0 ± 0.9	62.8 ± 0.9	65.9 ± 1.5	65.2 ± 0.9	62.3 ± 0.3

Table 7. Average training time per task (in hours) on MTIL. The total training time of CHEEM (1.45h) is split as: Supernet - 0.6h, Search - 0.53h, Finetuning - 0.32h.

CHEEM	CODA-P	L2P	Dual-P	SPrompts	DIKI	LoRA-CL
1.45	1.26	1.26	1.25	0.38	0.38	0.38

Table 6 & 7: Difficulty-awareness & Training overheads

Task-Difficulty Awareness (Table 6)

- MNIST and EuroSAT (ESAT) are assigned much lower FLOPs due to skip operations.
- Irregular, non-trivial network structures are successfully learned (e.g. MHSA-only blocks without FFN).

Reasonable Training Overhead (Table 7)

- NAS adds slight overhead compared to prompting baselines (L2P, CODA-P).
- However, it achieves much lower inference FLOPs and superior accuracy.

3. Comparative Analysis: Ablation Studies (Table 8 & 9)

Exploring the impact of placement choices and sampling strategies.

Table 8. Comparisons of two selected CHEEM placements.

Dataset	Method	ViT-Base			DEiT-Tiny		
		Avg. Acc	FLOPs	%Param	Avg. Acc	FLOPs	%Param
MTIL	MLP ^{Down}	85.88	62.31	0.40	74.51	4.47	6.32
	Attn Proj	85.57	65.91	0.25	74.39	4.42	1.79

Table 9. Comparisons of HEE and Uniform (i.e., Pure Exploration) sampling during Supernet training on MTIL benchmark.

Method	ViT-Base			DEiT-Tiny		
	Avg. Acc	FLOPs	%Param	Avg. Acc	FLOPs	%Param
HEE	85.88	62.31	0.25	74.51	4.47	6.32
Uniform	84.74	61.82	5.89	75.05	4.47	14.93

Table 8 & 9: Placements and Search comparisons

MLPDown vs. Projection (Table 8)

- Both achieve comparable accuracy.
- **MLPDown** yields better FLOPs reduction since skipping FFN blocks saves more computation than skipping Attention blocks.

HEE vs. Uniform Search (Table 9)

- Uniform search causes parameter explosion: **5.89%** growth (Uniform) vs. **0.25%** (HEE) on ViT-Base.
- **HEE-NAS** leverages task synergies to keep parameter growth parsimonious.

Conclusion & Core Takeaways

ExfCCL Solved

Decoupled continual memory allows accurate task-routing at inference without exemplars or task-ID inputs.

Difficulty-Aware NAS

HEE-NAS intelligently skips or adds components, optimizing the accuracy-efficiency trade-off under extreme imbalance.

"The smaller the backbone capacity, the greater the value of CHEEM's dynamic structure adaptation."